Automating Testing

Module 1
Introduction to Selenium

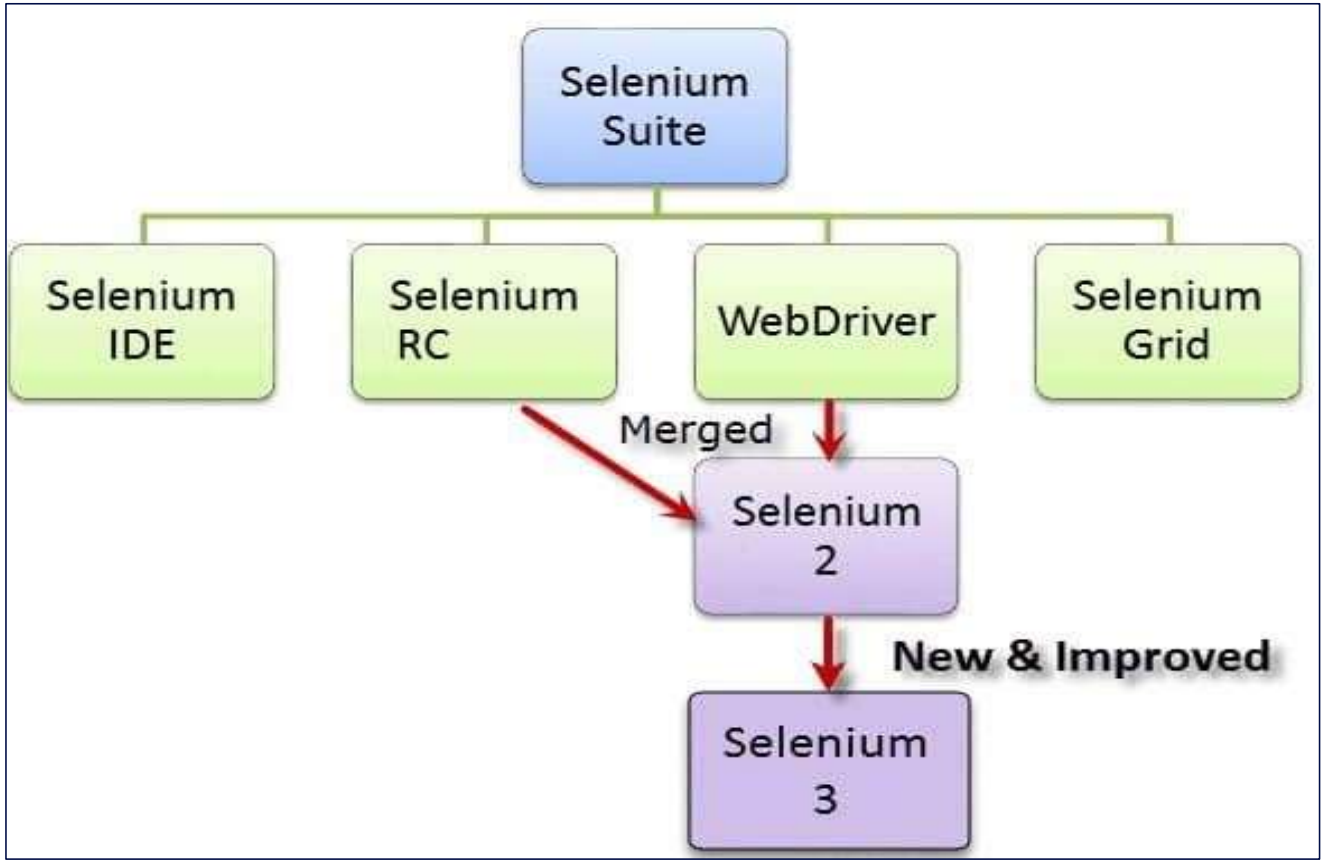
Agenda

- Selenium Automation Tool – Overview
- Selenium Versions and Special Components
- Selenium WebDriver Architecture
- Selenium Grid- Overview
- Setup Selenium Test Environment in IDE

Selenium Automation Tool- Overview

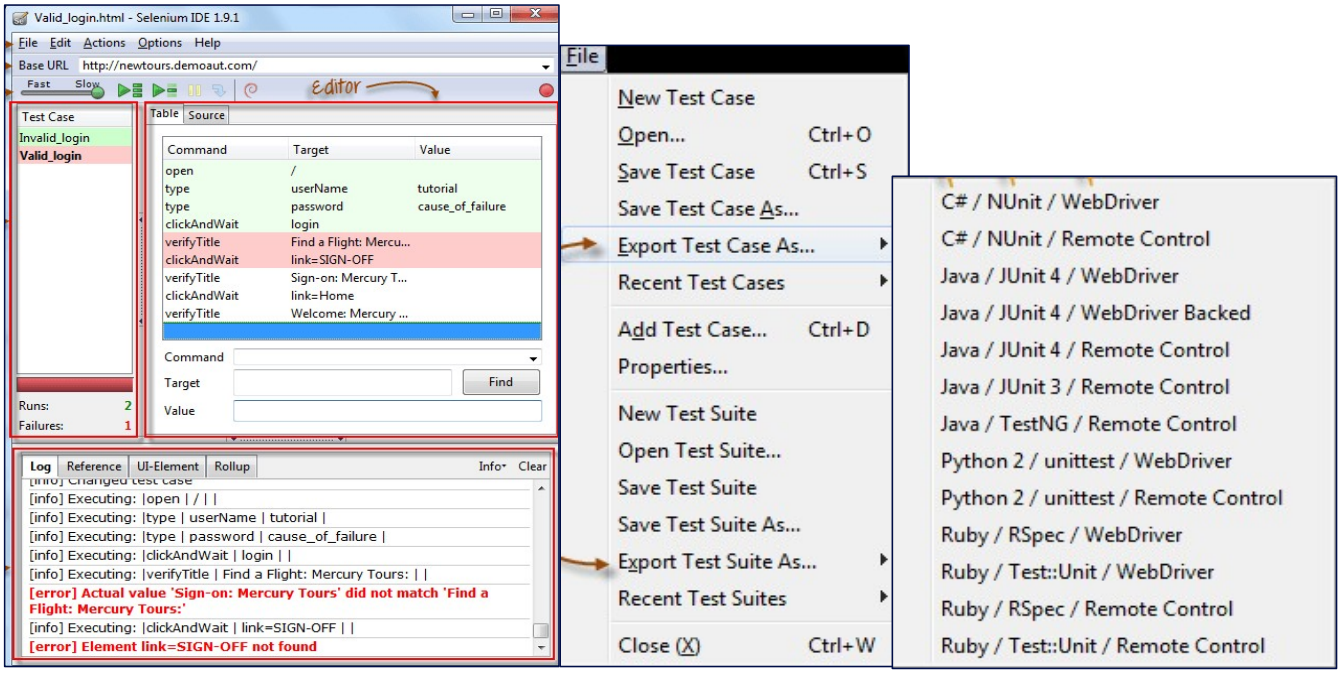
- **Open source**, **free to use**, and **free of charge.**
- Can run tests across **different browsers**
- Supports **various operating systems**
- Can execute tests **in parallel**
- Supports **mobile devices**

Selenium Versions



Selenium Integrated Development Environment[IDE]

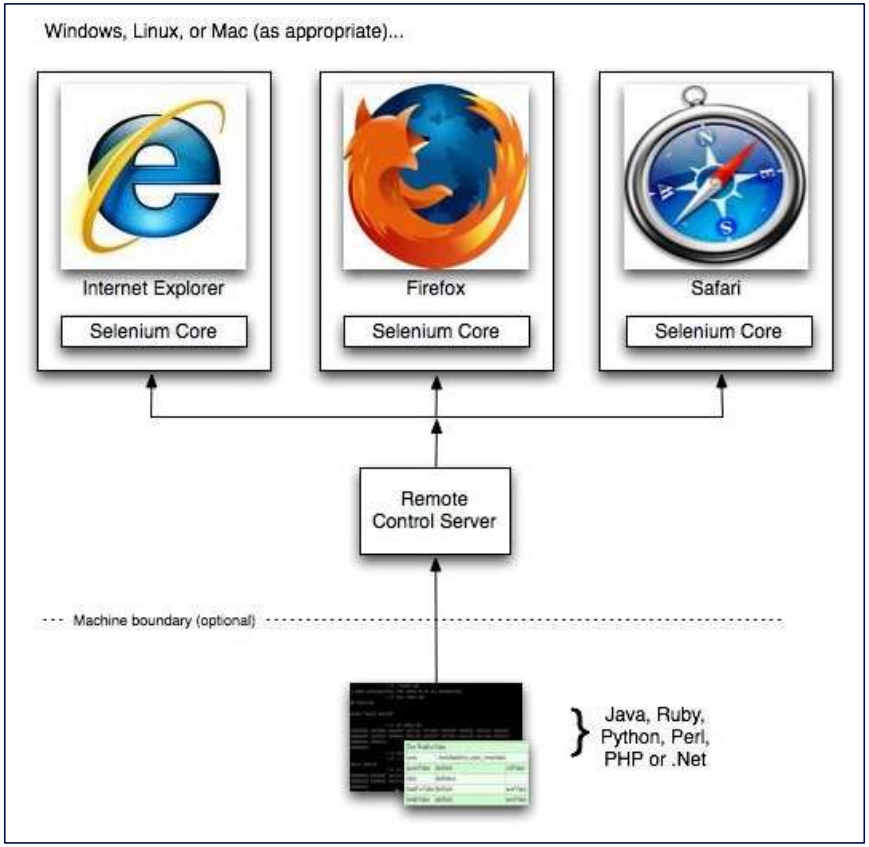
- Selenium IDE (Integrated Development Environment) is the simplest tool in the Selenium Suite.
- It is a Firefox add-on that creates tests very quickly through its record-and-playback functionality.
- Selenium IDE should only be used as a prototyping tool



Selenium Remote Control(RC)

- **Selenium Remote Control** (RC) is a test tool that allows you to write automated web application UI tests in any programming language against any HTTP website using any mainstream JavaScript-enabled browser.
- RC can support the following programming languages:
  - JavaScript
  - Java
  - C#
  - PHP
  - Python
  - Perl
  - Ruby

Selenium Remote Control(RC) Diagrammatic Representation

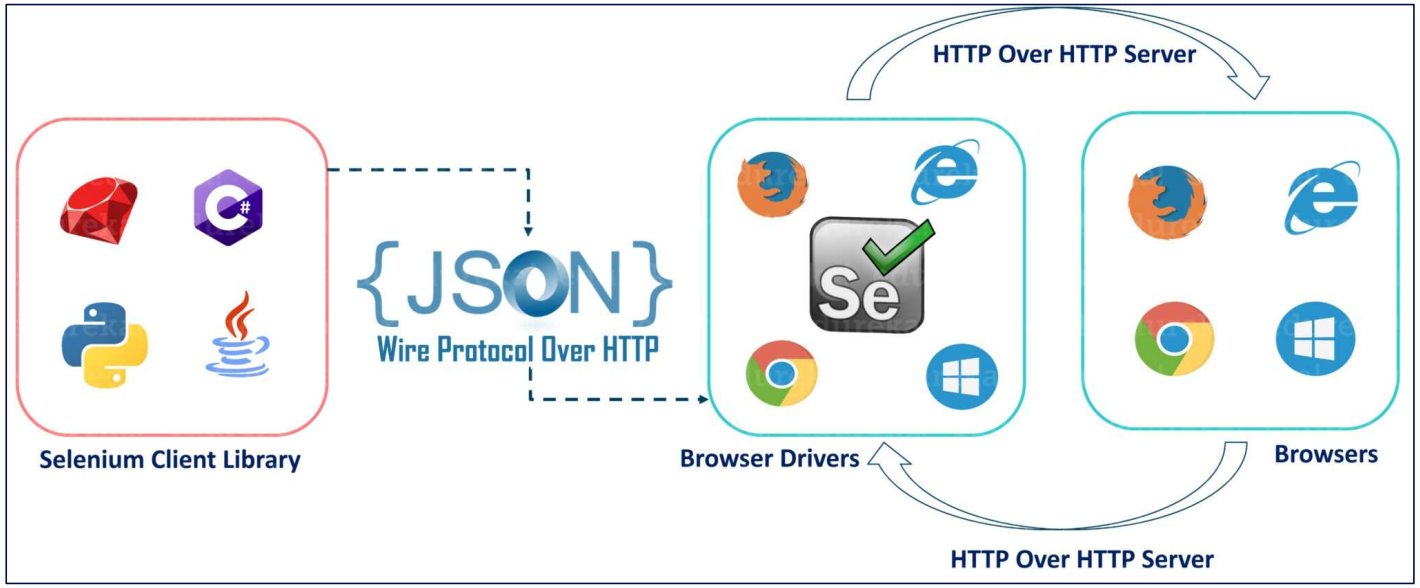


Selenium WebDriver

- WebDriver can support the following programming languages:
  - Java
  - C#
  - PHP
  - Python
  - Perl
  - Ruby

Selenium WebDriver Architecture

- Selenium WebDriver Architecture in detail. Architecture of Selenium WebDriver is all about how Selenium works internally.
- We know Selenium is a browser automation tool which interacts with browser and automate end to end tests of a web application.



Selenium Grid

- Selenium Grid is a part of the Selenium Suite that specializes in running multiple tests across different browsers, operating systems, and machines in parallel.
- **When to Use Selenium Grid?**
    - Run your tests against different browsers, operating systems, and machines all at the same time.
    - Save time in the execution of your test suites.

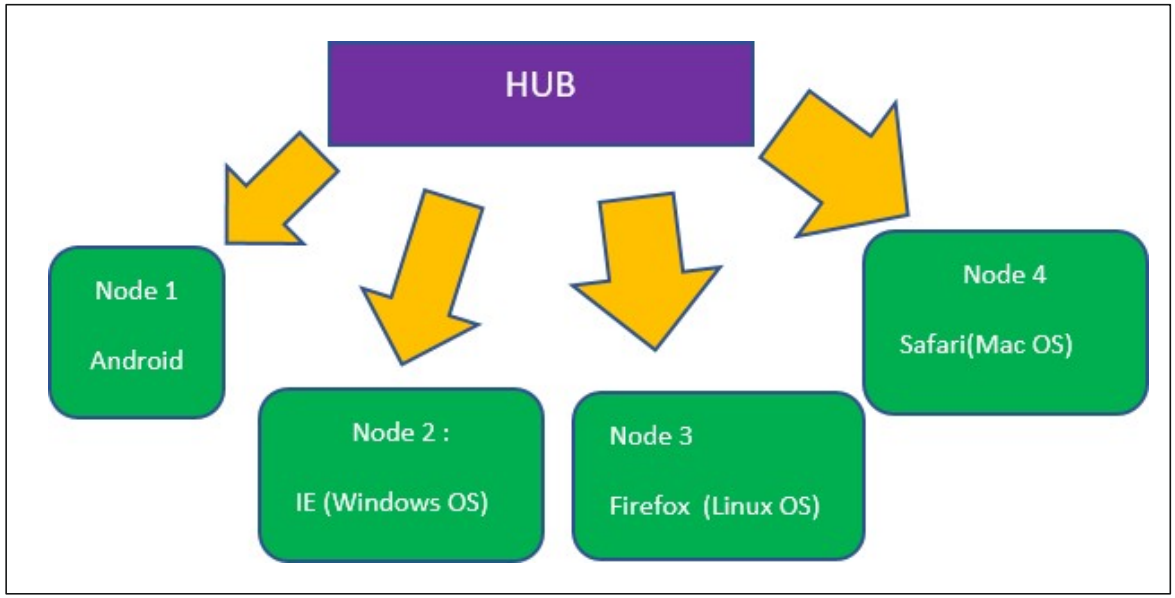
Selenium Grid Architecture

The Hub

- The hub is the central point where you load your tests into.

The Nodes

- Nodes are the Selenium instances that will execute the tests that you loaded on the hub.



Module 2
Handling Web Elements

Locators and its Types

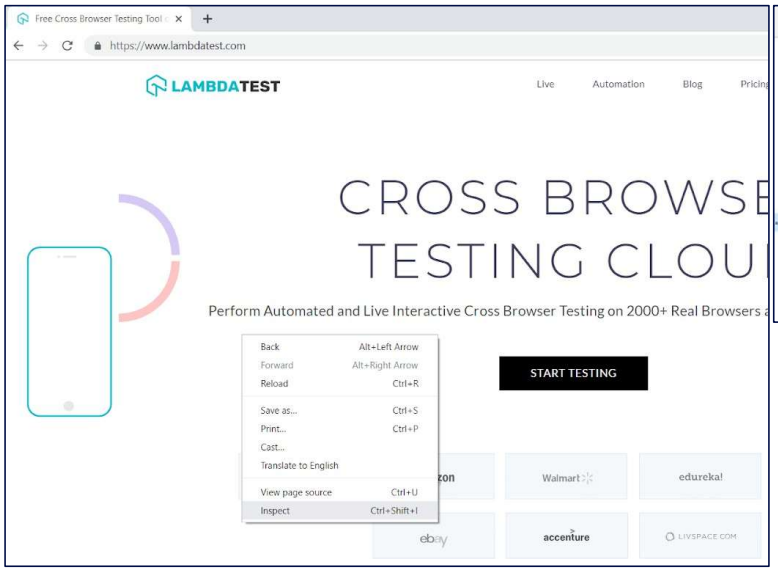
What are Locators?

Locators in Selenium are one of the most powerful commands. Its ideally the building block of the Selenium automation scripts.

1. ID
2. Name
3. Linktext
4. Partial Linktext
5. Tag Name
6. Class Name
7. CSS Selector
8. Xpath

Locators and its Types – Contd..

- Using the above locators in Selenium WebDriver you can locate elements through **“findElement/findElements”** syntax.
- let’s see how to find elements in DOM(Document object model).



Locators and its Types – Contd..

ID Locator:

- ID’s are unique for each element so it is common way to locate elements using ID Locator.
- ID locators are the fastest and safest locators out of all locators.

Syntax:

```
findElement(By.id("IdName"))
```

Name Locator:

- **Name** locator to identify the elements on our webpage. Locating elements using Name is same as locating elements using ID locator.

Syntax:

```
findElement(By.name("Name"))
```

Locators and its Types – Contd..

Class Name Locator :

Class Name locator gives the element which matches the values specified in the attribute name “class”.

Syntax:

```
findElement(By.className(“Element Class"))
```

Tag Name Locator:

Tag Name locator is used to find the elements matching the specified Tag Name. It is very helpful when we want to extract the content within a Tag.

Syntax:

```
findElement(By.tagName(“HTML Tag Name"))
```

Link Text Locator:

If there are multiple elements with the same link text then the first one will be selected. This Link Text locator works only on links (hyperlinks) so it is called as Link Text locator.

Syntax:

```
findElement(By.linkText(“LinkText"))
```

Locators and its Types – Contd..

PartialLink Text Locator:

- In some situations, we may need to find links by a portion of the text in a Link Text element. it contains. In such situations, we use Partial Link Text to locate elements.

Syntax:

```
findElement(By.partialLinkText(“PartialLinkText"))
```

CSS Selector Locator:

- Most of the automation testers believe that using CSS selector makes the execution of script faster compared to XPath locator. This locator is always the best way to locate elements on the page.

Tag and ID

Syntax:

```
findElement(By.cssSelector(tag#id))
```

Tag and Class

Syntax:

```
findElement(By.cssSelector(tag.class))
```

Tag and Attribute

Syntax:

```
findElement(By.cssSelector(tag[attribute=value]))
```

Locators and its Types – Cond..

Xpath Locator:

- XPath is designed to allow the navigation of XML documents, with the purpose of selecting individual elements, attributes, or some other part of an XML document for specific processing.

Syntax:

```
findElement(By.xpath (“Xpath"))
```

Xpath – Overview

Xpath :

- XPath is defined as **XML path**. **It is a syntax or language for finding any element on the web page using XML path expression**.
- XPath is used to find the location of any element on a webpage using HTML DOM structure.

Syntax: `Xpath=//tagname[@attribute='value']`

// : Select current node.
Tagname: Tagname of the particular node.
@: Select attribute.
Attribute: Attribute name of the node.
Value: Value of the attribute.

Selenium Form Web Element– Contd..

How to handle Text Box in Selenium:

To enter text into the Text Fields and Password Fields, sendKeys() is the method available on the WebElement.

Syntax: Reference_variable.sendKeys("value")

Reference_variable.clear()

How to handle Button in Selenium:

The buttons can be accessed using the click() method.

Syntax: Reference_variable.click()

Reference_variable.submit()

How to handle Radio Button in Selenium:

Radio Buttons too can be toggled on by using the click() method.

Syntax: radio.click()

radio.click()

Selenium Form Web Element– Contd..

How to handle Checkbox in Selenium:

Checkbox Buttons too can be toggled on by using the click() method.

- Boolean isSelected()
- Boolean isDisplayed()
- Boolean isEnabled()

Selenium Form Web Element– Contd..

How to handle Dropdown and Multiple Select Operations:

- We can be control Dropdown boxes using **Select** Option.
- Step1: Import the package - **org.openqa.selenium.support.ui.Select**

Select Methods:

- selectByVisibleText() and deselectByVisibleText()
- selectByValue() and deselectByValue()

Selenium Form Web Element– Contd..

Code:

```
dropdown.selectByIndex(4);
Thread.sleep(4000);
dropdown.selectByValue("ANTARCTICA");
dropdown.selectByVisibleText("ANTARCTICA");
Thread.sleep(5000);
```

Capturing the webpage screenshot in selenium

- Selenium can automatically take screenshots during execution.
- You need to type cast WebDriver instance to **TakesScreenshot**
- Taking Screenshot in Selenium is a 3 Step process:

Step 1: Convert web driver object to **TakeScreenshot.**

```
TakesScreenshot scrShot =((TakesScreenshot)webdriver);
```

Step 2: Call **getScreenshotAs** method to create image file

```
File  SrcFile = scrShot.getScreenshotAs(OutputType.FILE);
```

Step 3: Copy file to Desired Location

```
FileUtils.copyFile(SrcFile, DestFile);
```

Handling Alerts

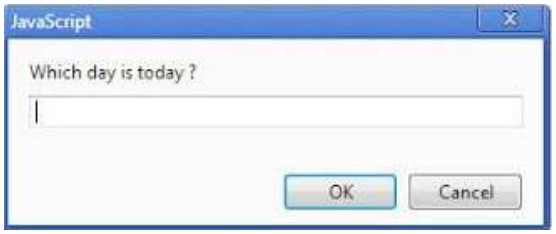
- Alert is a small message box which displays on-screen notification to give the user some kind of information or ask for permission to perform certain kind of operation.
- It may be also used for warning purpose. Here are few alert types:

Simple Alert:

This simple alert displays some information or warning on the screen.

Prompt Alert.

This Prompt Alert asks some input from the user and selenium webdriver can enter the text using sendkeys(" input…. ").



Handling Alerts- Contd..

Confirmation Alert:

This confirmation alert asks permission to do some type of operation.

Actions Class in Selenium

- Handling special keyboard and mouse events are done using the **Advanced User Interactions API**.
- It contains the **Actions** and the **Action** classes that are needed when executing these events.

The following are the most commonly used keyboard and mouse events provided by the Actions class.

- **clickAndHold()** - Clicks (without releasing) at the current mouse location.
- **contextClick()** - Performs a context-click at the current mouse location. (Right Click Mouse Action)
- **doubleClick()** - Performs a double-click at the current mouse location.
- **dragAndDrop(source, target)**- Performs click-and-hold at the location of the source element, moves to the location of the target element, then releases the mouse.

- **moveToElement(toElement)** - Moves the mouse to the middle of the element.
- **release()** - Releases the depressed left mouse button at the current mouse location

Actions Class in Selenium – Contd..

Step 1: Import the **Actions** and **Action** classes.

```
import org.openqa.selenium.interactions.Action;
import org.openqa.selenium.interactions.Actions;
```

Step 2: Instantiate a new **Actions** object.

```
Actions builder = new Actions(driver);
```

Step 3: Instantiate an **Action** using the **Actions** object

```
Action mouseOverHome = builder
        .moveToElement(link_Home)
        .build();
```

Step 4: Use the **perform()** method when executing the Action object

```
mouseOverHome.perform();
```

Handling Waits in Selenium

Why Do We need Waits in Selenium?

Most of the web applications are developed using Ajax and Javascript. When a page is loaded by the browser the elements which we want to interact with may load at different time intervals.

Selenium WebDriver waits:

Implicit Wait:

The implicit wait will tell to the web driver to wait for certain amount of time before it throws a "No Such Element Exception". The default setting is 0. Once we set the time, web driver will wait for that time before throwing an exception.

Syntax:

```
driver.manage().timeouts().implicitlyWait(TimeUnit.SECONDS);
```

Explicit Wait:

The explicit wait is used to tell the Web Driver to wait for certain conditions (**Expected Conditions**) or the maximum time exceeded before throwing an "**ElementNotVisibleException**" exception.

Syntax:

```
WebDriverWait wait= new WebDriverWait(WebDriverReference, TimeOut);
```

Handling Windows in Selenium

Handling Windows

Driver.getWindowHandles();

Driver.getWindowHandle();

Driver.getWindowHandles();

Syntax: `Set<String> reference_variable= driver.getWindowHandles()`

Driver.getWindowHandle();

Syntax: `String reference_variable= driver.getWindowHandle()`

Test Automation: Hands-on 1

1. Write a test to launch Edge Browser and print page title

```
package SELENIUM_CORE_TEST;

import org.openqa.selenium.WebDriver;

public class TC001 {


    //Global Variable - Accessible anywhere in this project
    public static  WebDriver driver;

    public static void main(String[] args) throws InterruptedException
    {

        System.setProperty("webdriver.edge.driver", "C:\\Users\\PSID\\msedgedriver.exe");
        driver=new EdgeDriver();
        driver.get("https://axess.sc.com/signin");
        System.out.println("Title: "+driver.getTitle());
        Thread.sleep(5000);
        driver.close();

    }

}
```

Test Automation: Hands-on 2

Write a test to verify if the page title contains Standard Chartered

```
public class TC002 {


    //Global Variable - Accessible anywhere in this project
    public static  WebDriver driver;

    public static void main(String[] args) throws InterruptedException
    {

        System.setProperty("webdriver.edge.driver", "C:\\Users\\PSID\\msedgedriver.exe");
        driver=new EdgeDriver();
        driver.manage().window().maximize();
        driver.get("https://axess.sc.com/signin");
        System.out.println("Title: "+driver.getTitle());
        driver.navigate().to("https://www.sc.com/in/");

        driver.navigate().back();
        driver.navigate().forward();
        driver.navigate().refresh();

        System.out.println(driver.getCurrentUrl());

        if (driver.getTitle().contains("Standard Chartered"))
        {
            System.out.println("PASS: Navigated to Desired Page");
        }
        else
        {
            System.out.println("FAIL: Navigated to Incorrect Page");
        }

        Thread.sleep(5000);
        driver.close();

    }

}
```

Test Automation: Hands-on 3

Write a Test to enter username and password and hit continue in the sign-in page.

```
package SELENIUM_CORE_TEST;

import org.openqa.selenium.By;

public class TC003 {


    //Global Variable - Accessible anywhere in this project
    public static  WebDriver driver;

    public static void main(String[] args) throws InterruptedException
    {

        System.setProperty("webdriver.edge.driver", "C:\\Users\\PSID\\msedgedriver.exe");
        driver=new EdgeDriver();
        driver.manage().window().maximize();
        driver.get("https://axess.sc.com/signin");
        System.out.println("Title: "+driver.getTitle());

        driver.findElement(By.id("user")).sendKeys("sam@bank.com");
        driver.findElement(By.id("pwd")).sendKeys("12345");
        Thread.sleep(3000);
        driver.findElement(By.className("submit-button")).click();
        Thread.sleep(3000);

        driver.close();

    }

}
```

Test Automation: Hands-on 4

Identify elements using x-path

```
package SELENIUM_CORE_TEST;

import org.openqa.selenium.By;

public class TC007 {


    //Global Variable - Accessible anywhere in this project
    public static  WebDriver driver;

    public static void main(String[] args) throws InterruptedException
    {

        System.setProperty("webdriver.edge.driver", "C:\\Users\\PSID\\msedgedriver.exe");
        driver=new EdgeDriver();
        driver.manage().window().maximize();
        driver.get("https://axess.sc.com/signin");
        System.out.println("Title: "+driver.getTitle());

        driver.findElement(By.id("user")).sendKeys("sam@bank.com");
        driver.findElement(By.id("pwd")).sendKeys("12345");
        Thread.sleep(3000);

        //Add the XPATH For View Password Element
        driver.findElement(By.xpath("//div[@role='button']")).click();
        Thread.sleep(3000);

        //Add the XPATH For Continue Element
        driver.findElement(By.xpath("//button[text()='Continue']")).click();
        Thread.sleep(3000);

        driver.close();

    }

}
```

Test Automation: Hands-on 5

Write a test to handle mouse events

```
public class TC009 {

    //Global Variable - Accessible anywhere in this project
    public static  WebDriver driver;
    public static Actions action;

    public static void main(String[] args) throws InterruptedException
    {

        TC009.AppStartup();
        TC009.PersonalLoans();
        TC009.CloseApp();
    }

    public static void AppStartup() throws InterruptedException
    {
        System.setProperty("webdriver.edge.driver", "C:\\Users\\PSID\\msedgedriver.exe");
        driver=new EdgeDriver();
        driver.manage().window().maximize();
        driver.get("https://www.sc.com/in/");
        //System.out.println("Title: "+driver.getTitle());
        Thread.sleep(4000);
    }
    public static void CloseApp()
    {
        driver.close();
    }
    public static void PersonalLoans() throws InterruptedException
    {
        //Mouse Events
        action=new Actions(driver);
        WebElement LoansMenu=driver.findElement(By.xpath("//button[text()='Loans']"));
        action.moveToElement(LoansMenu).build().perform();
        driver.findElement(By.xpath("//*[text()='Personal Loan']")).click();
        Thread.sleep(4000);
    }
}
```

Test Automation: Hands-on 6

Write a test to handle drop-down in Selenium

```
import org.openqa.selenium.By;

public class TC11 {

    public static void main(String[] args) throws InterruptedException
    {
        System.setProperty("webdriver.edge.driver", "C:\\Users\\PSID\\msedgedriver.exe");
        WebDriver driver=new EdgeDriver();
        driver.get("https://www.sc.com/in/loans/personal-loans/");
        Thread.sleep(4000);
        driver.findElement(By.xpath("//input[@aria-label='Loan amount'][1]")).clear();
        driver.findElement(By.xpath("//input[@aria-label='Loan amount'][1]")).sendKeys("500000");
        Thread.sleep(4000);
        String vEmi=driver.findElement(By.xpath("(//span[@class='sc-pil-calculator__payment'])[1]")).getText();
        System.out.println("EMI is : "+vEmi);
        System.out.println("PASS: Person Loan Calculator");
        Select select_tenure=new Select(driver.findElement(By.xpath("//select[@aria-label='For Yrs']")));

        select_tenure.selectByIndex(2);
        vEmi=driver.findElement(By.xpath("(//span[@class='sc-pil-calculator__payment'])[1]")).getText();
        System.out.println("EMI is : "+vEmi);
        Thread.sleep(4000);

        select_tenure.selectByValue("4");
        vEmi=driver.findElement(By.xpath("(//span[@class='sc-pil-calculator__payment'])[1]")).getText();
        System.out.println("EMI is : "+vEmi);
        Thread.sleep(4000);

        select_tenure.selectByVisibleText("5");
        vEmi=driver.findElement(By.xpath("(//span[@class='sc-pil-calculator__payment'])[1]")).getText();
        System.out.println("EMI is : "+vEmi);
        Thread.sleep(4000);

        driver.findElement(By.xpath("//span[text()='Apply Now']")).click();
        Thread.sleep(4000);
        driver.close();
    }
}
```

Thank You